

Time Complexity

Time complexity describes how the execution time of an algorithm increases with the size of the input.

We usually express it using **Big-O notation**, like:

- $O(1)$ – Constant time
- $O(\log n)$ – Logarithmic time
- $O(n)$ – Linear time
- $O(n \log n)$ – Linearithmic time
- $O(n^2)$ – Quadratic time

Example 1 – $O(1)$ (Constant Time)

```
public int getFirstElement(int[] arr) {  
    return arr[0];  
}
```

No matter how large the array is, it takes the same amount of time.

Example 2 – $O(n)$ (Linear Time)

```
public void printAllElements(int[] arr) {  
    for (int x : arr) {  
        System.out.println(x);  
    }  
}
```

Time increases linearly with the number of elements in the array. 

Example 3 – $O(n^2)$ (Quadratic Time)

```
public void printAllPairs(int[] arr) {
    for (int i : arr) {
        for (int j : arr) {
            System.out.println(i + ", " + j);
        }
    }
}
```

Two nested loops = time grows quadratically with input size.

$O(\log n)$ – Logarithmic Time Complexity

Concept:

- The input size is **reduced by a factor** (usually half) in each step.
- You don't process every element – instead, you "zoom in" on the target quickly.
- Think: **Divide and Conquer**.

Analogy:

Imagine you're looking for a word in a dictionary:

- You don't check every page.

- You open the middle, then decide whether to go left or right.
- That's **binary search** – and it's **logarithmic**.

Java Example – Binary Search ($O(\log n)$)

```
public int binarySearch(int[] arr, int target) {
    int left = 0;
    int right = arr.length - 1;

    while (left <= right) {
        int mid = left + (right - left) / 2;

        if (arr[mid] == target) {
            return mid;
        } else if (arr[mid] < target) {
            left = mid + 1;
        } else {
            right = mid - 1;
        }
    }

    return -1; // Target not found
}
```

Why $O(\log n)$?

- Suppose array length is 16 → You divide search space in half each time:
16 → 8 → 4 → 2 → 1 → done.
- That's at most $\log_2(16) = 4$ steps.
- So:
→ $O(\log n)$ steps,

→ Each step does **constant work**.

Efficient for large datasets. For 1 million items, it takes just ~20 steps.

$O(n \log n)$ – Linearithmic Time Complexity

Concept:

- This combines both **linear (n)** and **logarithmic ($\log n$)** growth.
 - Typically happens in efficient sorting algorithms.
 - You divide the problem ($\log n$), but also **touch every element (n)**.
-

Java Example – Merge Sort ($O(n \log n)$)

```
public void mergeSort(int[] arr) {
    if (arr.length < 2) return;

    int mid = arr.length / 2;
    int[] left = Arrays.copyOfRange(arr, 0, mid);
    int[] right = Arrays.copyOfRange(arr, mid, arr.length);

    mergeSort(left);
    mergeSort(right);
    merge(arr, left, right);
}

private void merge(int[] arr, int[] left, int[] right) {
    int i = 0, j = 0, k = 0;

    while (i < left.length && j < right.length) {
```

```

        arr[k++] = (left[i] < right[j]) ? left[i++] : right[j++];
    }

    while (i < left.length) arr[k++] = left[i++];
    while (j < right.length) arr[k++] = right[j++];
}

```

Why $O(n \log n)$?

- $\log n$ → Number of times you split the array in half (divide step).
- n → Number of elements you need to merge at each level.
- So:
 - Each level of recursion does $O(n)$ work,
 - There are $\log n$ levels.

Total time:
 $O(n \log n)$

Java Shortcut – Built-in Sorting (Also $O(n \log n)$)

```
Arrays.sort(arr);
```

Under the hood (for primitives), Java uses **Dual-Pivot QuickSort** which is optimized to $O(n \log n)$ on average.

Comparing $O(\log n)$ vs $O(n \log n)$

Complexity	Meaning	Example
------------	---------	---------

$O(\log n)$	Input shrinks every step, minimal work	Binary Search
$O(n \log n)$	Input shrinks, but each level touches all n	Merge Sort, QuickSort

Complexity	Common Use Case	Example
$O(\log n)$	Searching in sorted data	Binary Search
$O(n \log n)$	Efficient sorting	Merge Sort, Arrays.sort

Space Complexity

Space complexity describes how much memory an algorithm uses in relation to the input size.

It also uses **Big-O notation**.

Example 1 – $O(1)$ Space

```
public int sumList(int[] arr) {
    int total = 0;
    for (int x : arr) {
        total += x;
    }
    return total;
}
```

Uses a single variable regardless of the list size → Constant space.

Example 2 – $O(n)$ Space

```
public int[] copyList(int[] arr) {  
    int[] newArr = new int[arr.length];  
    for (int i = 0; i < arr.length; i++) {  
        newArr[i] = arr[i];  
    }  
    return newArr;  
}
```